

TECHNICAL ARCHITECTURE • V1.0

Technical Requirements Document (TRD)

System Architecture, Multi-Tenancy Engine & Security Blueprint

PROJECT

Gymholik SaaS

VERSION

1.0.0

AUTHOR

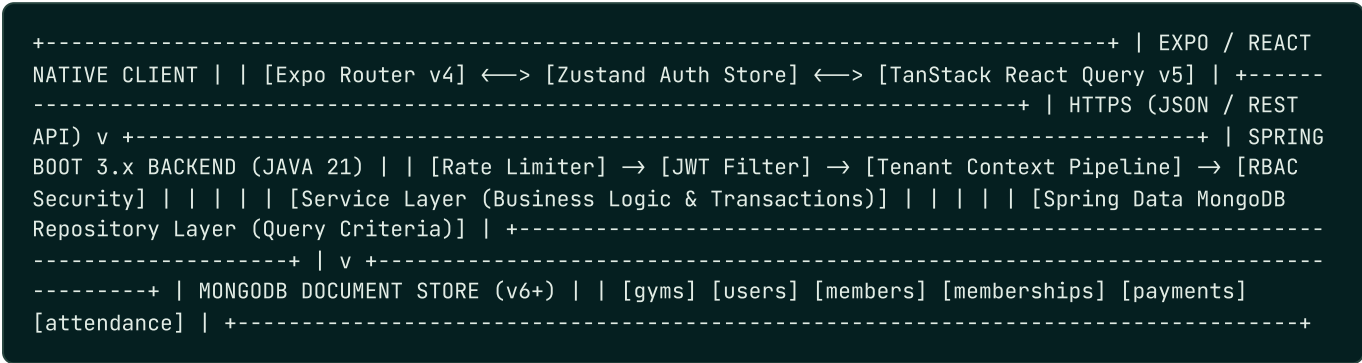
Engineering Team

DATE

October 2026

1. Architecture Overview & System Topology

Gymholik implements a modern, decoupled client-server architecture. The mobile client is a cross-platform React Native / Expo application communicating over HTTPS with a high-performance Spring Boot 3 RESTful API backed by a scalable MongoDB document database.



2. Technology Stack Breakdown

Tier	Technology	Version	Architectural Rationale
Backend Runtime	Java (LTS)	21	Virtual threads (Project Loom), pattern matching, modern memory efficiency.
Web Framework	Spring Boot	3.2.x	Industry-standard enterprise DI, security integrations, rapid REST development.
Security Engine	Spring Security	6.2.x	Stateless JWT authentication, method-level @PreAuthorize RBAC.
Database	MongoDB	6.0+	Schema flexibility for varied gym membership models, fast document indexing.
Mobile Runtime	React Native / Expo	SDK 52	Universal iOS/Android compilation, EAS build pipeline, OTA updates.
Mobile Routing	Expo Router	v4	File-based navigation with deep-link support and dynamic tab layout shielding.
State & Cache	Zustand + TanStack Query	v5	Zero-boilerplate global auth store with automatic query caching and deduplication.

3. Multi-Tenancy Architecture & Data Isolation

Gymholik utilizes a **Shared Database, Shared Schema** multi-tenancy model partitioned via a required `gymId` foreign key on every tenant-scoped document.

3.1 The TenantContext Pipeline

1. Every authenticated request carries a Bearer JWT containing `gymId`, `userId`, and `role` in its claims.
2. `JwtFilter` parses the token, authenticates the user principal, and extracts `gymId`.
3. `TenantFilter` establishes a `ThreadLocal` context via `TenantContext.setGymId(gymId)`.
4. Service layers query repositories with criteria pre-filtered by `TenantContext.getGymId()`.
5. In `finally` block of filter, `TenantContext.clear()` prevents thread reuse contamination.

```
public class TenantContext { private static final ThreadLocal<String> CURRENT_GYM = new ThreadLocal<>(); public static void setGymId(String gymId) { CURRENT_GYM.set(gymId); } public static String getGymId() { return CURRENT_GYM.get(); } public static void clear() { CURRENT_GYM.remove(); } }
```

4. Security Architecture & Threat Modeling

4.1 Authentication & Token Lifecycle

- **Access Token:** Short-lived HMAC-SHA256 JWT (expires in 30 minutes). Contains identity and role payload.
- **Refresh Token:** Cryptographically secure UUID stored in MongoDB (expires in 30 days). Evaluates token rotation upon exchange.
- **Brute-Force Rate Limiting:** Memory-backed sliding window rate limiter (`LoginRateLimiter`) blocks IP addresses after 5 failed login attempts per minute.

4.2 Privacy & Data Masking

To adhere to privacy standards and prevent competitor staff poaching:

- Member mobile numbers are masked for staff unless explicitly permitted by gym settings: `+91 98****1211`.
- Gym secret codes are visible exclusively to verified gym owners.

5. CI/CD & Deployment Strategy

Backend Containerization

Mobile Distribution

Packaged via multi-stage Dockerfile based on Eclipse Temurin Alpine 21. Orchestrated using Docker Compose with MongoDB container and volume persistence.

Compiled via Expo Application Services (EAS Build). Produces optimized Android AAB for Google Play and iOS IPA for Apple TestFlight / App Store.